

Local Repository Migration Plan

Migrating in-flight branches to Knitweb/pulse

The legacy remote febuzz/knitweb has been archived and superseded by the Knitweb organization repository Knitweb/pulse. This document describes the current local state, the target state, and the step-by-step procedure to rebase, validate, and open pull requests for all in-flight work.

Prepared by: Kimi Code CLI

Date: 2026-06-23

Target remote: `git@github.com:Knitweb/pulse.git`

1. Current State Inventory Knitweb/pulse

The local repository was originally cloned from the archived remote febuzz/knitweb. During the last session the remote was updated to Knitweb/pulse and the main branch was reset to the migrated upstream head. Several feature branches and a stash of unrelated work remain based on the old history and must be reconciled.

1.1 Active Local Branches

- feat/lens-rlm-digest - MeTTa-inspired atomspace + KnitwebLensAdapter for virtualpc LLM agents (1 commit).
- feat/pouw-quorum-settlement - verifier quorum and quorum-aware escrow settlement (2 commits).
- feat/anchor-origintrail-backend - placeholder branch containing stashed anchor/Rosetta work in progress.

1.2 Stashed Work

- Stash: anchor-rosetta-wip - uncommitted files including anchor/__init__.py, anchor/origintrail.py, Rosetta Code loom, GitHub templates, CONTRIBUTING.md, and PROCESS_MANAGEMENT.md.
- This work predates the migration and must be reviewed for relevance before rebasing.

1.3 Divergence from Upstream

Upstream main (Knitweb/pulse) is many commits ahead of the old febuzz/knitweb main. The repository has grown from ~40 source files to more than 1,100 property tests. New modules that overlap with local work include:

- knitweb/interpret/ - content-addressed agent memory nodes (SkillNode, ProjectNode, etc.) and distillation.
- knitweb/pouw/quorum.py - a full BFT supermajority quorum already exists, so the local quorum module may duplicate or complement it.
- knitweb/lens/ - an empty package directory exists; the local lens implementation can land here.

2. Migration Procedure Internal migration planning document - do not distribute

The migration is performed branch-by-branch. Each branch is rebased onto the new upstream main, conflicts are resolved in favour of upstream semantics, tests are run, and the branch is force-pushed to Knitweb/pulse.

2.1 Prerequisite: Update Local References

```
cd repo/knitweb
git remote set-url origin git@github.com:Knitweb/pulse.git
git fetch origin
git checkout main
git reset --hard origin/main
```

2.2 Rebase Each Feature Branch

```
# Lens branch
git checkout feat/lens-rlm-digest
git rebase main
# resolve conflicts; prefer upstream APIs
PYTHONPATH=src python3 -m pytest tests/property/test_lens.py -q
git push --force-with-lease -u origin feat/lens-rlm-digest
```

```
# POWW quorum settlement branch
git checkout feat/pouw-quorum-settlement
git rebase main
# Expect overlap with upstream knitweb/pouw/quorum.py
PYTHONPATH=src python3 -m pytest tests/property/test_quorum_settlement.py -q
git push --force-with-lease -u origin feat/pouw-quorum-settlement
```

2.3 Decide on Stashed Work

- Inspect the stashed files against upstream main to detect duplication (e.g., anchor/origintrail.py versus existing OriginTrail integration).
- Split the stash into coherent topic branches: feat/github-templates, feat/anchor-origintrail, feat/loom-rosettacode.
- Discard anything already present in upstream or rendered obsolete by the new interpret/ lens foundation.

3. Conflict Resolution Rules knitweb/pulse

Because the upstream repository evolved independently after the archive, some local files will conflict. Apply the following rules in order:

- Upstream API wins. If upstream already provides an equivalent module, drop the local implementation and adapt callers.
- Preserve tests. A local test that exercises upstream behavior is valuable even if the implementation is replaced.
- Rename on collision. If the local module has different semantics (e.g., MeTTa atomspace versus interpret/distill), rename or nest it under a distinct subpackage such as knitweb/lens/metta.
- Keep pure-Python and dependency-free. Do not introduce new third-party dependencies that conflict with upstream's stdlib-only policy.

The local feat/pouw-quorum-settlement branch defines knitweb.pouw.quorum, which already exists upstream with additional validation (ABSTAIN verdict, strict majority enforcement). Do not overwrite upstream quorum.py. Instead, rebase so that the local commit only adds quorum_settlement.py and its tests, re-using upstream Verdict/Outcome/tally.

4. Validation Gate

Before any branch is pushed, the following checks must pass:

- Feature tests pass: PYTHONPATH=src python3 -m pytest tests/property/test_<feature>.py -q
- Full property suite passes (subset if slow WebRTC tests are present): PYTHONPATH=src python3 -m pytest tests/property -q -k 'not webrtc'
- No type regressions: python3 -m mypy src/knitweb/<changed_package> if mypy is available.
- LOC report refreshed: python3 tools/loc_report.py

The upstream test suite contains ~1,157 tests. Running the full suite may take several minutes due to WebRTC transport tests. Use targeted tests during active development and run the full suite only as a final gate.

5. Pull Request and Feedback Loop

After a branch is pushed to Knitweb/pulse, open one pull request per feature. Follow the repository's existing PR template and reference the originating archived repository only for historical context.

```
gh pr create --base main --head feat/lens-rlm-digest \
  --title "feat(lens): MeTTa-inspired atomspace for virtualpc agents" \
  --body "Rebases #<legacy-PR> onto Knitweb/pulse. Adds ..."
```

- Assign or request review from Claude / the maintainer team.
- Monitor PR comments using `gh pr view <number> --comments` or the GitHub web UI.
- Implement feedback in-place on the same branch; do not open a second PR for the same feature unless requested.
- Never merge - per workflow instructions, merges are handled by Claude after changes are solid.

6. Risk Assessment and Rollback

- Risk: Rebase introduces silent semantic changes because upstream APIs moved. Mitigation: run feature tests and inspect diff against main after rebase.
- Risk: Local quorum module conflicts with upstream. Mitigation: delete local quorum.py and adapt tests to upstream types.
- Risk: Stashed work is partially obsolete. Mitigation: review each file before rebasing; discard duplicates.
- Risk: Force-push overwrites shared history. Mitigation: use `--force-with-lease` and confirm no one else has pushed to the feature branch.

7. Rollback Procedure

```
# If a rebase goes wrong, abort or reset
git rebase --abort
git checkout <branch>
git reset --hard <pre-rebase-backup-ref>
# Or recreate from the original commit before migration
git reflog | head -20
```

8. Post-Migration Checklist

- All local branches pushed to Knitweb/pulse with green CI.
- Open PRs exist for feat/lens-rlm-digest and feat/pouw-quorum-settlement.
- Stashed work split into coherent branches or discarded.
- Local main tracks origin/main exactly.
- Archived febuzz/knitweb remote removed from local remotes.